Presented by PAVITRA PATHAK
PRN: 25070521066

Dijkstra's Algorithm

Finding the shortest path in a weighted graph

Introduction to Dijkstra's Algorithm

Definition

Dijkstra's algorithm finds the shortest path from a single source vertex to every other vertex in a weighted graph with non-negative edge weights.

The shortest-path problem

Given a weighted graph and a source vertex, find the minimum-total-weight route to every other vertex, like the quickest way between two cities.

Why is it greedy?

At each step it permanently picks the closest unvisited vertex. With non-negative weights this local choice is always globally optimal, so no decision is revisited.

Key facts

1956: Conceived by Edsger W. Dijkstra (published 1959)

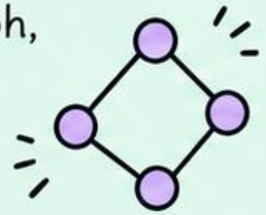
Greedy: Locally optimal choice at every step

≥ 0: Edge weights must be non-negative

Key Terms

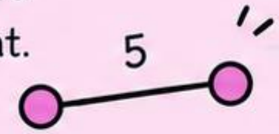
Vertex (Node)

A point in the graph, such as a city or router.



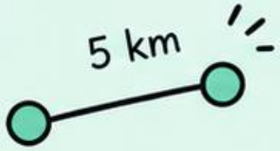
Edge

A connection between two vertices with a cost or weight.



Weight

The distance, time, or cost assigned to an edge.



Shortest Path

The route with the minimum total weight between two vertices.



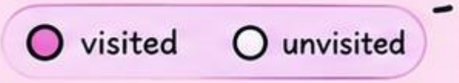
Relaxation

Update a neighbour's distance if a shorter route is found.



Visited / Unvisited

Track which vertices have been processed and which are yet to be explored.



How Dijkstra's Algorithm Works

Initialize

dist[src] = 0,
others = ∞

Select

Pick unvisited node u with the smallest dist

Mark visited

dist[u] is now final

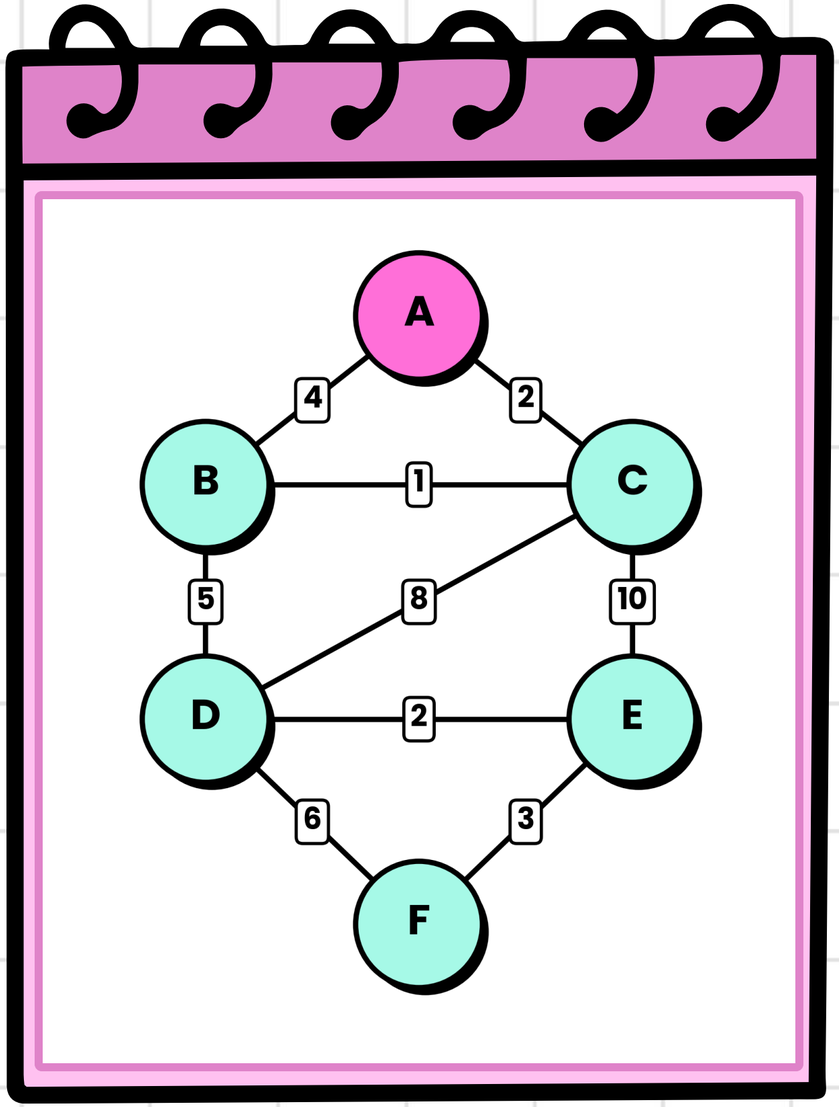
Relax neighbours

For each edge (u, v) update dist[v] if shorter

The relaxation step

```
if dist[u] + w(u,v) < dist[v]:  dist[v] = dist[u] + w(u,v)
```

- A shorter route to v through u has been found, so update its distance and remember u as the predecessor.
- Unvisited nodes left? Yes: repeat from Select. No: done, the shortest paths are found.

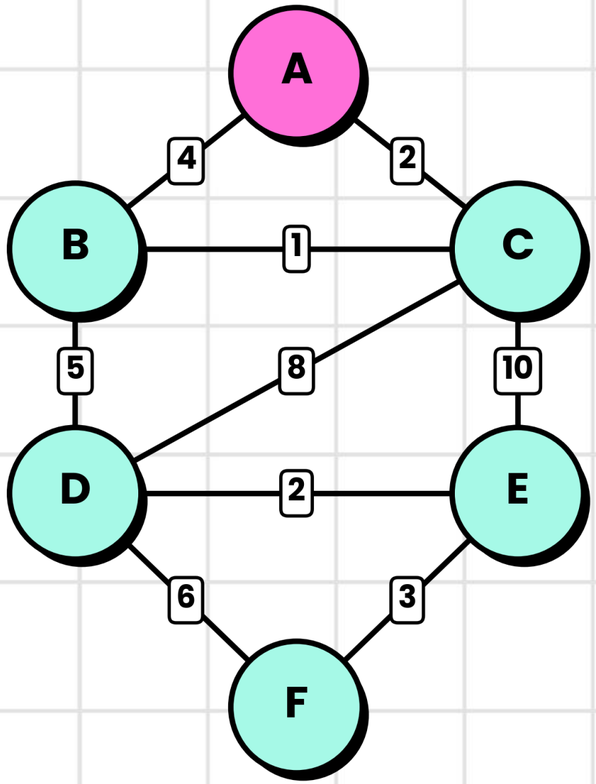


Example: Problem Statement

Find the shortest distance from source A to every other vertex of this weighted, undirected graph.

6 vertices, 9 edges, all weights positive. Source A is highlighted.

Step~by~Step Example

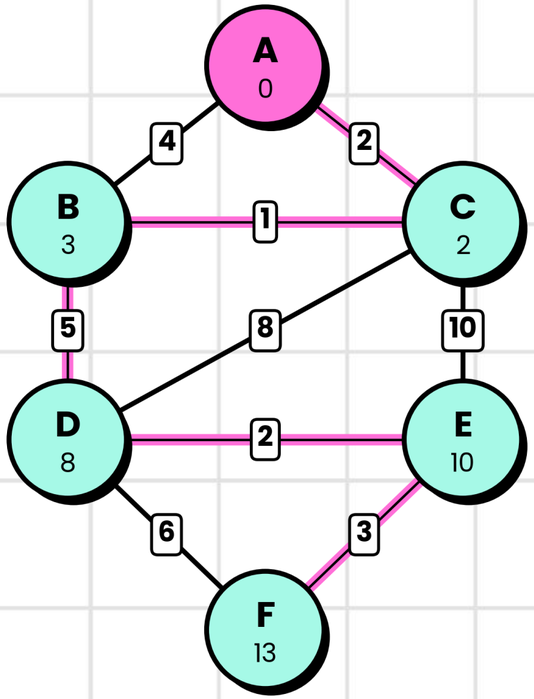


Source = A (highlighted)

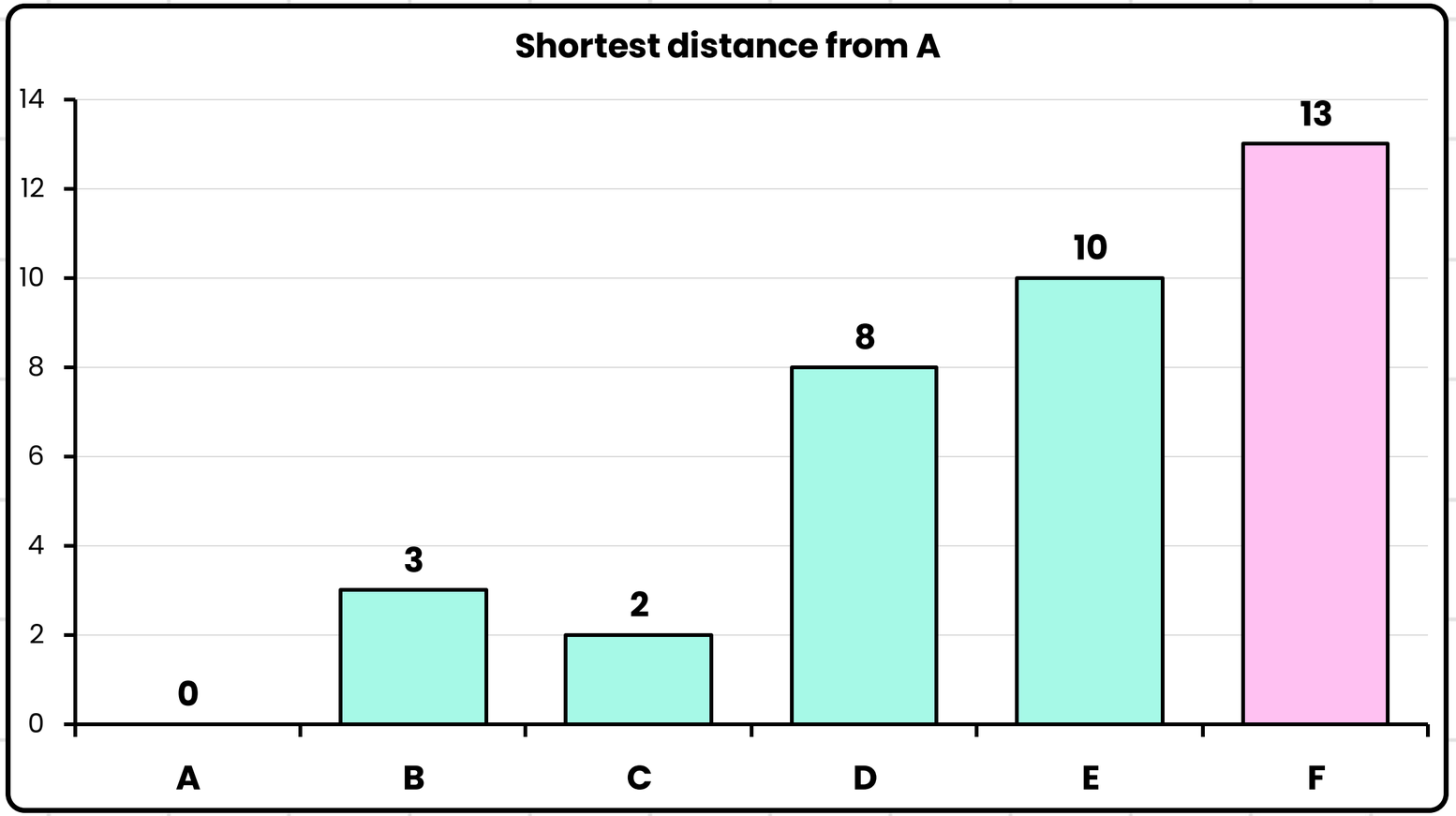
Step	Visit	A	B	C	D	E	F
Init	–	0	∞	∞	∞	∞	∞
1	A	0	4	2	∞	∞	∞
2	C	0	3	2	10	12	∞
3	B	0	3	2	8	12	∞
4	D	0	3	2	8	10	14
5	E	0	3	2	8	10	13
6	F	0	3	2	8	10	13

Highlighted cells were updated in that step

Final Result: Shortest Paths from A



Highlighted edges = shortest paths



Shortest path A → F: A → C → B → D → E → F **total cost = 13**

Pseudocode & Time Complexity

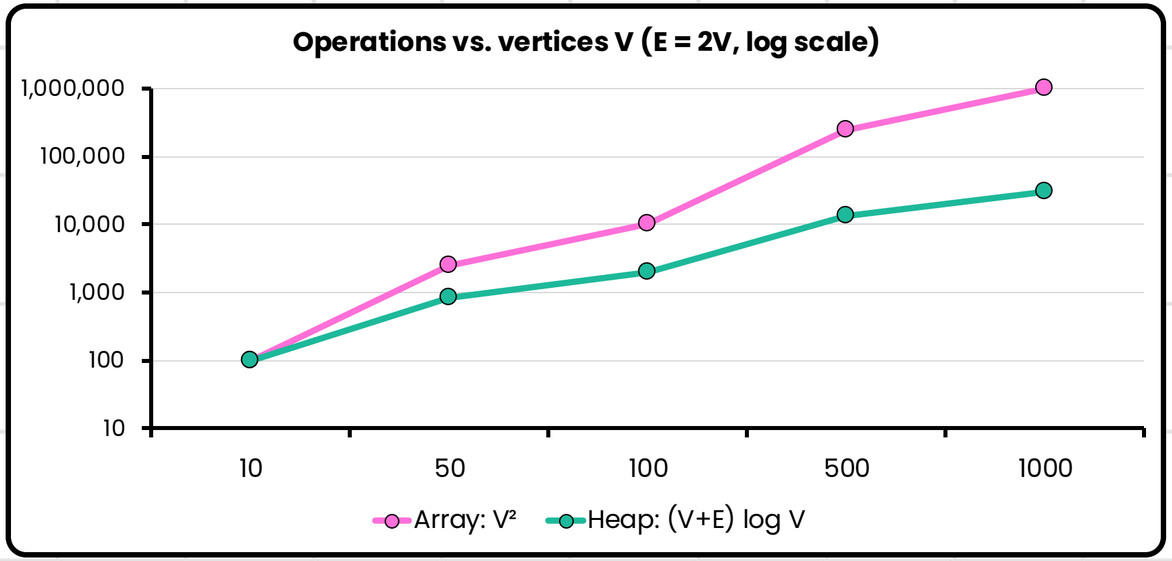
PSEUDOCODE

```
function Dijkstra(Graph, source):
  for each vertex v:
    dist[v] = INF;  prev[v] = NULL
  dist[source] = 0
  Q = priority queue of all vertices
  while Q is not empty:
    u = vertex in Q with min dist[u]
    remove u from Q
    for each neighbour v of u:
      alt = dist[u] + w(u, v)
      if alt < dist[v]:
        dist[v] = alt;  prev[v] = u
  return dist, prev
```

dist[v]: best known distance
prev[v]: predecessor

Implementation	Time complexity
Array / adjacency matrix	$O(V^2)$
Binary heap (priority queue)	$O((V + E) \log V)$
Fibonacci heap	$O(E + V \log V)$

Space complexity: $O(V)$ for dist[], prev[] and the visited set



Real-World Applications

Navigation

Finds the fastest or shortest route over road networks using distance or travel-time weights.

- Maps & GPS apps

Networking

Link-state routing protocols compute the least-cost path to every router.

- OSPF, IS-IS

Logistics

Plans delivery and warehouse routes that minimise distance, fuel and time.

- Couriers, supply chains

Robotics

Plans collision-free paths across grid or occupancy maps.

- Warehouse robots, drones

Games

Pathfinding for characters across maps where terrain has different costs.

- NPC movement

Large systems use faster variants built on the same idea, such as A* search and contraction hierarchies.

Advantages and Limitations

Advantages

- Guarantees optimal paths (weights ⩾ 0)
- Works on directed and undirected graphs
- Fast with a priority queue
- Gives paths to all vertices at once
- Simple to implement and widely used

Limitations

- Fails with negative edge weights
- Explores in all directions, no heuristic
- Must recompute if the graph changes
- Needs the whole graph in memory
- Slow on huge dense graphs

Conclusion

Dijkstra's algorithm repeatedly settles the closest unvisited vertex, giving guaranteed shortest paths on graphs with non-negative weights. It underpins modern routing and pathfinding.

References

Cormen et al., Introduction to Algorithms, MIT Press
E. W. Dijkstra, “A note on two problems in connexion with graphs”, 1959
Sedgewick and Wayne, Algorithms, 4th ed.

THANK YOU

Created using:~ Claude AI